Day 2 — Building the First REST Endpoint

Goal: Ship a working HTTP endpoint that returns job applications as JSON. **Result:** `GET /api/v1/applications` returns three seeded rows from H2. **End-to-end touched:** Java entity → JPA persistence → Spring Data repository → REST controller → Jackson JSON serialization.

---

1. The Layered Architecture

Almost every Spring Boot backend (and every backend at FAANG) is built in three layers. Today we built a thin slice through all three.

Layer	What it is	Today's class
Entity	A Java class that maps to a database table. One row in the table = one Java object.	`JobApplication`
Repository	A class that knows how to read/write entities to the database. You don't write SQL — Spring Data JPA generates it.	`JobApplicationRepository`
Controller	A class that handles HTTP requests. Maps URLs → Java methods → JSON responses.	`JobApplicationController`

Data flow when a browser hits `/api/v1/applications`:

```
Browser → Controller → Repository → H2 database
                                        ↓
Browser ← Controller ← Repository ← rows
         (Jackson auto-converts to JSON)
```

Why this order matters: we built entity first because the repository needs to know *what* it stores, and the controller needs to know *what* it returns. Bottom-up: data shape → data access → HTTP exposure.

2. Layer 1 — The Entity

What an entity is

An **entity** is a Java object; a **row** is data on disk (or in H2's memory). They are two representations of the same thing in two different worlds.

	Row (database world)	Entity (Java world)
Lives in	A table on disk (or H2's memory)	RAM, as a Java object
Looks like	`1, Google, SWE, APPLIED`	`JobApplication{id=1, company="Google"…}`
Spoken to via	SQL (`SELECT * FROM job_application`)	Java method calls (`app.getCompany()`)

JPA is the translator between them. `save(entity)` → SQL `INSERT`. `findById(1)` → SQL `SELECT`.

Mental model: Row = data at rest in the DB. Entity = same data, in Java's hands so your code can manipulate it.

The code

```
package com.kuldeep.jobtrail;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class JobApplication {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String company;
    private String role;

    @Enumerated(EnumType.STRING)
```

```
    private Status status;
}
```

Annotations explained

Annotation	Purpose
`@Entity`	"Instances of this class map to rows in a table." Without this, JPA ignores the class. Table name defaults to snake-case → `job_application`. Override with `@Table(name="...")`.
`@Id`	Marks the primary key field. **Every entity must have exactly one.**
`@GeneratedValue(strategy = IDENTITY)`	"Don't ask me what the id is — let the database assign it on insert." `IDENTITY` = use DB's auto-increment column.
`@Enumerated(EnumType.STRING)`	Store the enum's name (`"APPLIED"`) in the DB, not its position.

Lombok annotations

Annotation	What it generates
`@Getter`	`getId()`, `getCompany()`, `getRole()`, `getStatus()`
`@Setter`	`setId(...)`, `setCompany(...)`, etc.
`@NoArgsConstructor`	`public JobApplication() {}` — **JPA requires this** to instantiate entities via reflection
`@AllArgsConstructor`	`public JobApplication(Long, String, String, Status)` — handy for tests/seed data

Trade-off (interview answer): Lombok hides what's actually compiled. Some shops ban it for that reason. Most use it because the boilerplate savings are huge.

Key design decisions

Why `Long` (boxed) instead of `long` (primitive)? `Long` can be `null` — which is what we want **before** the row is saved. The DB hasn't assigned an id yet. A primitive `long` defaults to `0`, falsely implying "this entity has id 0." `Long` over `Integer` because `int` overflows past ~2.1 billion rows.

Why `EnumType.STRING` and not `ORDINAL`? `ORDINAL` stores the enum's position as an integer (`SAVED=0, APPLIED=1, ...`). If you reorder the enum from `SAVED, APPLIED, ...` to `APPLIED, SAVED, ...`, every existing row's status flips silently — no error, no log, data corruption. `STRING` stores `"APPLIED"` literally — reordering is harmless. **Always use `STRING` for enum columns.** Classic JPA footgun and a frequent interview question.

FAANG trivia: `jakarta.persistence` vs `javax.persistence` The package was renamed when Oracle handed Java EE to the Eclipse Foundation. Spring Boot 3.x uses `jakarta`; Spring Boot 2.x used `javax`. If you see `javax.persistence` in a tutorial, it's old.

`GeneratedValue` strategies (interview-relevant)

Strategy	How it works	When
`IDENTITY`	DB auto-increment column	Default for MySQL, H2, Postgres SERIAL
`SEQUENCE`	Separate DB sequence object	Postgres native, Oracle
`AUTO`	JPA picks for you	Convenient but hides the choice
`UUID`	Java-generated UUIDs	Distributed systems, no central DB

What Hibernate does at startup

Because `spring.jpa.hibernate.ddl-auto=update` is set, Hibernate inspects the entity at startup and issues:

```
CREATE TABLE job_application (
    id      BIGINT       NOT NULL AUTO_INCREMENT,
    company VARCHAR(255),
    role    VARCHAR(255),
    status  VARCHAR(255),
    PRIMARY KEY (id)
);
```

`ddl-auto=update` only **adds** columns/tables — it never drops anything. In production you replace it with a real migration tool (Flyway or Liquibase). We'll do that in Phase 3.

---

3. Layer 2 — The Repository

What a Spring Data JPA repository is

An **interface** that says "I want CRUD operations on this entity." You don't write the implementation — Spring Data JPA writes it at runtime.

The code

```
package com.kuldeep.jobtrail;

import org.springframework.data.jpa.repository.JpaRepository;

public interface JobApplicationRepository extends JpaRepository<JobApplication, Long> {
}
```

That's the **whole file**. Five lines.

How the magic works

By extending `JpaRepository<JobApplication, Long>`, your interface inherits all of `JpaRepository`'s methods — for free:

```
JobApplication save(JobApplication app);              // INSERT or UPDATE
Optional<JobApplication> findById(Long id);           // SELECT WHERE id = ?
List<JobApplication> findAll();                       // SELECT *
long count();                                         // SELECT COUNT(*)
void deleteById(Long id);                             // DELETE WHERE id = ?
boolean existsById(Long id);                          // SELECT 1 WHERE id = ?
// ... ~15 more
```

You wrote zero of these. Spring generates them.

How Spring "finds" the interface

At startup, Spring Boot:

1. Scans the classpath for interfaces extending `JpaRepository`.
2. For each one, builds a **dynamic proxy class** that implements the interface.
3. Wires that proxy as a singleton bean in the Spring context.
4. Makes it injectable wherever you need it.

Two terms to know for interviews: **classpath scanning** + **dynamic proxy generation**.

Generic type parameters

Parameter	Meaning
`JobApplication`	The entity type this repository manages
`Long`	The type of that entity's `@Id` field

The second parameter must match the `@Id` field's type. Mismatch = startup failure.

Method-name query derivation (preview for Day 3+)

Add a method declaration like:

```
List<JobApplication> findByCompany(String company);
```

…and Spring Data parses the **method name**:

- `findBy` → generate `SELECT`
- `Company` → matches the `company` field
- The `String` param binds to `WHERE company = ?`

Generated SQL: `SELECT * FROM job_application WHERE company = ?;`

Other keywords: `findByCompanyAndStatus`, `findByStatusOrderByIdDesc`, `countByStatus`, `deleteByCompany`. For queries too complex for a method name, use `@Query("...")` with explicit JPQL or SQL.

Why this matters (interview articulation)

Without Spring Data JPA, every project re-writes the same DAO boilerplate:

```
public class JobApplicationDao {
    @PersistenceContext private EntityManager em;

    public JobApplication save(JobApplication a) {
        if (a.getId() == null) { em.persist(a); return a; }
        return em.merge(a);
    }
    public JobApplication findById(Long id) { return em.find(JobApplication.class, id); }
    public List<JobApplication> findAll() {
        return em.createQuery("SELECT a FROM JobApplication a", JobApplication.class).getResultList
    }
    // ... 15 more methods, copy-pasted across every entity
}
```

Spring Data JPA replaces all that with one interface. **Less code = fewer bugs.**

---

4. Layer 3 — The Controller

What a REST controller is

The class that translates **HTTP ↔ Java method calls**. The only layer that knows the project is an HTTP API at all.

- **Incoming:** an HTTP request → controller picks the right Java method and calls it.
- **Outgoing:** the method returns a Java object → controller serializes it to JSON.

The code

```
package com.kuldeep.jobtrail;

import java.util.List;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/applications")
public class JobApplicationController {
```

```
    private final JobApplicationRepository repository;

    public JobApplicationController(JobApplicationRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<JobApplication> getAll() {
        return repository.findAll();
    }
}
```

Annotations explained

Annotation	Purpose
`@RestController`	`@Controller` + `@ResponseBody`. Method return values become JSON in the response body.
`@RequestMapping("/api/v1/applications")`	The base path for every endpoint in this class. All methods live underneath.
`@GetMapping`	"This method handles `GET` requests at the class's base path." Sibling annotations: `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`.

`@Controller` vs `@RestController`

	`@Controller` (alone)	`@RestController`
Returns	View names (HTML templates)	Response body (JSON)
Use for	Server-rendered HTML (Thymeleaf, JSP)	REST APIs

For a JSON backend, **always** use `@RestController`.

Constructor injection (and why)

We injected the repository via the **constructor**, not via `@Autowired` on a field.

	Constructor injection (what we did)	Field injection
Immutability	`final` field — set once, never reassigned	Can never be `final`
Testability	`new JobApplicationController(fakeRepo)` — no Spring context needed	Requires reflection or Spring context
Missing deps	Compile/startup error — fails fast	Hidden until runtime usage

Spring's official recommendation since 4.x: constructor injection. Field injection is a tutorial smell.

Jackson serialization

When the controller returns `List<JobApplication>`, Spring hands it to **Jackson** (the JSON library bundled with `spring-boot-starter-web`). Jackson:

1. Reflects on each object to find getter methods.
2. Derives JSON field names from getter names (`getCompany` → `"company"`).
3. Recursively serializes values.

This is why our entity needed `@Getter` — without getters, Jackson sees no fields and emits `{}`.

Sample output:

```
[
  {"id": 1, "company": "Google", "role": "SWE",              "status": "APPLIED"},
  {"id": 2, "company": "Meta",   "role": "SWE",              "status": "INTERVIEWING"},
  {"id": 3, "company": "Amazon", "role": "Backend Engineer", "status": "SAVED"}
]
```

Customization options exist (`@JsonProperty`, `@JsonIgnore`, custom serializers) but defaults Just Work for plain POJOs.

URL versioning convention

The `/api/v1/` prefix is intentional:

- **`/api/`** — separates JSON endpoints from any future static pages or server-rendered HTML.

- `/v1/` — lets you ship `/v2/applications` later (with breaking schema changes) without breaking existing clients.

Other versioning approaches: header-based (`Accept: application/vnd.jobtrail.v1+json`), query-string (`?version=1`). URL-path versioning is the most common — most visible, easiest to debug. Versioning is a frequent system-design interview topic.

---

5. Bonus — Seed Data with `CommandLineRunner`

Without this, the endpoint returns `[]` (H2 starts empty every restart).

```
package com.kuldeep.jobtrail;

import java.util.List;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DataSeeder {

    @Bean
    CommandLineRunner seedJobApplications(JobApplicationRepository repository) {
        return args -> {
            if (repository.count() > 0) return;
            repository.saveAll(List.of(
                new JobApplication(null, "Google", "SWE",              Status.APPLIED),
                new JobApplication(null, "Meta",   "SWE",              Status.INTERVIEWING),
                new JobApplication(null, "Amazon", "Backend Engineer", Status.SAVED)
            ));
        };
    }
}
```

Annotation / type	Purpose
`@Configuration`	"This class produces beans." Spring scans `@Bean` methods inside.
`@Bean`	Register the return value as a Spring-managed bean.

Annotation / type	Purpose
`CommandLineRunner`	A Spring interface with one method, `run(String... args)`. **Any bean of this type runs once after Spring finishes wiring but before serving traffic.** Perfect for seed data, sanity checks, warm-ups.
`if (repository.count() > 0) return;`	Defensive guard — only seed when the table is empty. Doesn't matter today (H2 wipes on restart) but matters when Postgres takes over.
`new JobApplication(null, ...)`	The `null` is the `id` — let the database assign it. Recall: this is exactly why `id` had to be `Long` (boxed), not `long`.

---

6. End-to-End Request Flow

When a browser hits `GET http://localhost:8080/api/v1/applications`:

1. **Tomcat** (the embedded web server inside Spring Boot) accepts the TCP connection on port 8080.
2. **Spring DispatcherServlet** receives the HTTP request.
3. It matches the URL `/api/v1/applications` + method `GET` to `JobApplicationController.getAll()`.
4. It calls `getAll()`, which calls `repository.findAll()`.
5. The Spring Data **proxy** intercepts `findAll()`, generates `SELECT * FROM job_application`, runs it against H2.
6. H2 returns row data → Hibernate maps each row into a `JobApplication` Java object.
7. The list is returned to the controller, then to the DispatcherServlet.
8. **Jackson** serializes the list to JSON.
9. Tomcat writes the JSON bytes back over the TCP connection with `Content-Type: application/json` and HTTP `200 OK`.
10. Browser displays the JSON.

You wrote about 30 lines of code total. Everything else is framework.

---

7. Interview Anchors

If a recruiter asks "tell me about a project you've built", these are the high-leverage talking points from Day 2:

1. **Layered architecture** — entity / repository / controller, with separation of concerns.
2. **Spring Data JPA** — interface-only repositories, dynamic proxy generation, method-name query derivation.
3. **Constructor injection** — why it's preferred over field injection (immutability, testability, fail-fast).
4. **Enum mapping with `EnumType.STRING`** — and the data-corruption risk of `ORDINAL`.
5. **`Long` vs `long` for primary keys** — null safety for unsaved entities.
6. **API versioning via URL prefix** — `/api/v1/`.
7. **Jackson auto-serialization** — getter-based, no manual JSON code.
8. **`ddl-auto=update`** — convenient for dev, dangerous for prod (use Flyway/Liquibase instead).

---

8. What's Next (Day 3 preview)

- **`POST /api/v1/applications`** — create a new job application.
- **`GET /api/v1/applications/{id}`** — fetch one by id (introduces `@PathVariable` and `Optional` handling).
- **DTOs vs Entities** — why you should not return your entity directly to the API in real apps.
- **`@Service` layer** — where business logic lives.

Files created today

```
src/main/java/com/kuldeep/jobtrail/
├── JobApplication.java              ← entity
├── Status.java                      ← enum
├── JobApplicationRepository.java    ← repository
├── JobApplicationController.java    ← REST controller
└── DataSeeder.java                  ← seed data
docs/
├── notes/day-2.md                   ← this file
└── qna/day-2.md                     ← Q&A learning log
scripts/
└── notes-to-pdf.sh                  ← reusable md → PDF converter
```